OPERATING SYSTEMS – ASSIGNMENT 1

TASK 3 – Evaluate Operating-System Structures (4 Marks)
CO1–CO5 | Bloom's Level: K5 – Evaluate

Introduction

An operating-system structure describes how the kernel and its services are organised. The hospital must select a structure that balances reliability, performance, security and ease of updating.

1. Comparison of operating-system structures

Simple / monolithic structure: Most OS services run together in kernel space. It can be fast because components communicate directly, but a fault in one component or driver can affect the whole system.
Layered structure: The OS is divided into layers. Each layer uses the services of the layer below it. This makes the design easier to understand, test and maintain, but communication through layers can add overhead.
Microkernel structure: Only essential functions such as basic scheduling, memory management and inter-process communication remain in the kernel. Other services run as separate user-space servers. This improves fault isolation and security, but message passing can add performance overhead.
Modular structure: The kernel supports loadable modules, such as device drivers and file-system modules. Modules can be added or removed when needed. It gives flexibility and good performance, although a faulty kernel module can still affect the kernel.
Hybrid structure: It combines ideas from monolithic, microkernel and modular designs to balance performance and modularity.

2. Effect on reliability, performance and updates

A monolithic kernel usually provides good performance but has a larger trusted kernel area. A layered design improves organisation and maintenance. A microkernel provides better isolation because many services run outside the kernel, but inter-process communication may take extra time. A modular kernel makes it easier to add or update features, but kernel modules must be carefully tested. A hybrid kernel attempts to combine performance with modular organisation.

3. Suitable structures for hospital environments

Bedside patient monitors – Microkernel / small real-time kernel: Patient monitors require predictable operation and high reliability. Keeping the kernel small reduces the amount of code running with full privileges. Separate services can improve fault isolation. The chosen real-time OS must also be validated to meet the 10 ms alarm deadline.
Central database server – Modular or hybrid server kernel: A modular or hybrid structure supports high performance, file systems, networking and storage drivers. It is suitable for a database server that handles many users and large amounts of data. High availability should additionally be provided through a cluster; kernel structure alone does not provide fail-over.

4. Examples: Linux, Windows, Android and real-time OS

Linux: Commonly described as a monolithic but modular kernel because it supports loadable kernel modules.
Windows: Uses a hybrid kernel design with kernel-mode and user-mode components.
Android: Uses the Linux kernel with Android-specific framework and system services above it.
FreeRTOS: A small real-time kernel intended for embedded systems. **QNX:** Uses a microkernel architecture and is designed for reliability and real-time use.

5. Alternative concept: monolithic kernel for bedside monitors

If a bedside monitor used a monolithic kernel instead of a microkernel, services such as drivers and file systems would run in kernel space. A system-call request may have lower communication overhead because services can be called directly within the kernel. A microkernel may need message passing between separate servers, which can add context switches and communication time. However, the exact time depends on implementation and hardware.
A monolithic kernel may contain more services in privileged memory. A microkernel keeps the kernel smaller, although total system memory also includes user-space servers. Most importantly, a faulty driver in a monolithic kernel can crash or corrupt the whole system. In a microkernel design, a driver running as a separate service may fail without bringing down the entire OS, if recovery is supported.

6. Final evaluation

A microkernel-based real-time OS is a strong choice for bedside monitors because reliability and fault isolation are important. A modular or hybrid server OS is suitable for the database server because it offers performance and flexible support for hardware and services. The hospital should test the actual system under its expected workload before deployment.

Conclusion

The best OS structure depends on the environment. A small and fault-isolated structure suits patient monitors, while a high-performance modular or hybrid structure suits the database server.

Prepared from the corresponding task in the uploaded CSA04 Operating Systems Assignment 1 document. Add your name, register number, required screenshots and your own measured evidence wherever requested before submission.